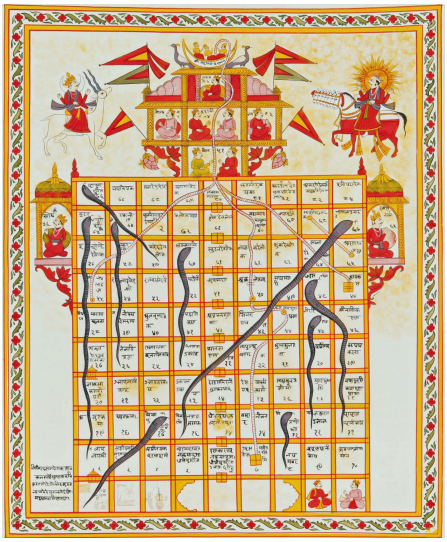
# 6th Assignment



Originating in India as **Moksha Patam**, Snakes and Ladders is a classic board game that relies on pure luck when rolling a die. The board features numbered squares, with players moving their pieces based on die rolls. Some squares contain snakes that send players backward, while others have ladders that propel them forward.

The classic version of **Chutes and Ladders** as published by **Milton Bradley** in 1943 consists of 100 squares arranged in a 10x10 grid. It adds 9 ladders and 10 chutes, and is played with a 6-sided die (or spinner). Play starts outside the board, and players roll the die to move their piece. Landing on the bottom of a ladder allows the player to move up to the top of the ladder, while landing on the head of a snake sends the player down to its tail.

## Snakes and Ladders Simulator

Implement a simulator for the game **Snakes and Ladders** using **Markov Chain Monte Carlo** (MCMC) methods.

The simulator should allow for the following features:

- **Game Setup**
  - The board should consist of an $N \times M$ grid of squares, with the default size being $10 \times 10$.
  - The board should accomodate an arbitrary number of snakes and ladders.
    - No snake or ladder should start or end on the same square as itself or any other snake or ladder.
    - No snake or ladder should start at the last square.
- **Game Rules**
  - The game is played with an $n$-sided die, with the default being a 6-sided die.
    - (optional) Allow for non-uniform die distributions.
  - The game is won when either the last square is reached exactly, or when a player rolls a die that would exceed the last square.
    - If the game must be won by reaching the last square exactly, any roll that would overshoot the last square does not move the player.
- **Game Simulation**
  - The program simulates games of Snakes and Ladders by repeatedly rolling the die and advancing the position according to the game rules.
  - The simulation is run for a specified number of iterations, and the results are collected.
    - (optional) Run simulations in parallel.
- **Game Statistics**
  - The program should calculate and display statistics about the game:
    - The average number of rolls to win the game.
    - The shortest number and sequence of rolls that lead to a win.
    - The number and relative frequency of each snake and ladder being traversed.

## Specific Requirements

- Implement the game board as a **graph**, where each square is represented by a node.
- Allow for non-interactive use of the program, e.g. using command line arguments or configuration files.

- Individual simulations should abort after a certain number of steps (e.g. in an infinite loop) even if no win condition is encountered.
- Prepare multiple board and game configurations (ideally with interesting properties) and compare their results.

## Common Requirements

- The program should be implemented in C.
  - The code should be well structured into header and source files.
  - Any public functions should be well documented, private functions should be commented where necessary.
  - The code should be compiled using `clang` with the "`-Wall -Wextra -Werror`" flags.
  - The application MUST not crash or leak memory.
- The assignment MUST be handed in through the provided GitHub Classroom assignment.
  - Assignments pushed after the deadline will be marked as late and will incur a point penalty.
  - Only push plain-text files, but no binaries, archives, or build artifacts.